KUVEMPU UNIVERSITY

Jnanasahyadri, Shankaraghatta, Shivamogga

PROJECT REPORT ON

"Backtracking: Graph Colouring and Sum of Subsets"

Submitted By:

Spoorthi G Kudtarkar	[U06SR25S0053]
Amulya	[U06SR25S0081]
Kavitha M	[U06SR25S0160]
Varun Sagar H L	[U06SR25S0082]

"Submitted in Partial Fulfillment of the Requirements for The Mini Project in Bachelor of Computer Application."

Under the Guidance of

Mr. PRADEEP G S

Asst. Professor

Department of BCA

SRNMNC, SHIMOGA

S.R.N.M.N College of Applied Sciences

Department of Bachelor Of Computer Application

2026-2027

S.R.N.M.N College of Applied Sciences

Balraj Urs Road, Shivamogga

CERTIFICATE

Certified that the project entitled **"Backtracking: Graph Colouring and Sum of Subsets"** carried out by Spoorthi G Kudtarkar, Amulya, Kavitha M and Varun Sagar H L, bonafide students of S.R.N.M.N College of Applied Sciences, in partial fulfilment of the requirements for the Bachelor of Computer Application by Kuvempu University, Shankaraghatta, during the year 2026–2027. This mini project was completed as part of the coursework on Design and Analysis of Algorithm of Python Programming.

REVIEWER	GUIDE	HOD
Mr. Pradeep G S	Mr. Pradeep G S	Mr. H S Dattatri
Asst. Professor	Asst. Professor	HOD
Dept. of BCA	Dept. of BCA	Dept. of BCA
SRNMNC, Shimoga	SRNMNC, Shimoga	SRNMNC, Shimoga

Title of the Project: Backtracking (Graph Colouring and Sum of Subsets)

Introduction

Backtracking is a general algorithmic technique for solving problems that require choosing a sequence of decisions from a set of options, subject to constraints. It builds a solution one step at a time and abandons ("backtracks" from) a partial solution as soon as it is certain that it cannot lead to a valid complete solution. It can be seen as an improved, pruned form of brute-force search over the *state space tree* of a problem.

This project implements two classic backtracking problems in Python: **Graph (m-)Colouring**, which assigns at most *m* colours to the vertices of a graph so that no two adjacent vertices share a colour, and the **Sum of Subsets** problem, which finds all subsets of a given set of positive integers whose elements add up to a given target. The time complexity of both is analysed.

Key Components

- **1. State Space Tree:** A tree in which each node represents a partial solution and each level represents one decision.
- **2. Solution (Candidate) Vector:** The array being built, e.g. colors[v] for graph colouring, or the include/exclude choices for subsets.
- **3. Constraint (Feasibility) Function:** Checks whether the current partial solution is valid, e.g. no adjacent vertex has the same colour.
- **4. Bounding Function:** Prunes branches that cannot reach a solution, e.g. stop when the running sum exceeds the target.
- **5. Promising Node:** A node that may still lead to a valid solution; non-promising nodes are pruned.
- **6. Backtracking Step:** Undoing the last choice and trying the next alternative when a dead end is reached.
- **7. Termination Condition:** The search ends when a solution is found (or all solutions are collected) or the whole tree has been explored.

Workflow

- **1. Start** with an empty partial solution at the root of the state space tree.
- **2. Choose** the next option for the current decision level.
- **3. Check** feasibility with the constraint and bounding functions.
- **4. Extend** the solution and recurse to the next level if the choice is promising.
- **5. Backtrack** (undo the choice) if the choice fails or the recursion returns without success.
- **6. Record** the solution when all decisions are made validly, then stop or continue for more solutions.

Benefits

- **Simplicity:** Naturally expressed with recursion and easy to implement.
- **Efficiency over brute force:** Pruning removes large parts of the search space.
- **Generality:** One template solves many constraint-satisfaction problems.
- **Completeness:** Can find one solution or all solutions.

Challenges

- **Exponential worst case:** Running time can still grow exponentially with input size.
- **Quality of pruning:** Performance depends heavily on good bounding functions and choice ordering.
- **Recursion depth:** Deep recursion may use significant stack memory for large inputs.
- **Not suited for optimisation at scale:** Large instances may need heuristics or dynamic programming.

Applications

- **Graph Colouring:** Exam timetabling, register allocation in compilers, map colouring, frequency assignment.

- **Sum of Subsets:** Resource and budget allocation, cryptography (knapsack-based schemes), partitioning problems.
- **Other:** N-Queens, Sudoku solving, Hamiltonian cycle, maze solving, puzzle solving.

Future Trends

- **Constraint propagation:** Combining backtracking with forward checking and arc consistency.
- **Parallel backtracking:** Exploring subtrees of the state space tree on multiple cores or GPUs.
- **Learning-guided search:** Using machine learning to choose variable and value ordering heuristics.

Complexity Analysis

Problem	Time Complexity (worst case)	Space Complexity
Graph m-Colouring	$O(m^V \cdot V)$	O(V)
Sum of Subsets	$O(2^n)$	O(n)

Graph colouring: there are m^V possible colour assignments for V vertices, and each feasibility check costs O(V). Pruning reduces the practical running time considerably, but the worst case stays exponential. The recursion depth is V, so the space used is O(V).

Sum of subsets: each of the n elements is either included or excluded, giving a binary state space tree with up to 2^n leaves. Sorting the numbers and pruning when the running sum exceeds the target cuts many branches; the worst case is still $O(2^n)$ with O(n) recursion depth.

Advantages and Disadvantages of Backtracking

Advantages

- **Easy to implement:** The recursive choose–check–undo structure is simple to write and understand.

- **Better than brute force:** Pruning discards partial solutions that cannot succeed, so far fewer candidates are examined.
- **Finds all solutions:** It can return one valid solution or every valid solution, as sum of subsets does.
- **Memory efficient:** Only the current path of the state space tree is stored, so space use is O(depth).
- **Widely applicable:** The same template solves many problems, such as graph colouring, N-Queens, Sudoku and Hamiltonian cycle.

Disadvantages

- **Exponential worst case:** Time can still grow as $O(m^V)$ or $O(2^n)$ when little pruning is possible.
- **Slow on large inputs:** It becomes impractical for big problem sizes compared with dynamic programming or greedy methods.
- **Depends on ordering and bounds:** Poor choice ordering or weak bounding functions reduce the benefit of pruning.
- **Recursion overhead:** Deep recursion adds function-call cost and can cause stack overflow.
- **Repeated work:** It does not remember earlier results, so the same sub-problems may be solved many times.

Conclusion

Backtracking is a systematic and powerful technique for solving constraint-satisfaction and combinatorial problems. By building solutions incrementally and abandoning dead ends early, it avoids the full cost of brute-force enumeration. The graph colouring and sum of subsets implementations in this project show how the same choose–check–recurse–undo template applies to different problems, and the complexity analysis shows that, although pruning helps greatly in practice, the worst-case cost remains exponential. The project gives a clear understanding of recursion, state space trees and pruning strategies.

Graph Colouring

Introduction

Graph colouring is the problem of assigning colours to the vertices of a graph so that no two adjacent vertices, that is, vertices joined by an edge, have the same colour. In the **m-colouring problem**, a graph with V vertices and a number m are given, and the task is to decide whether the vertices can be coloured using at most m colours. The smallest number of colours that is enough for a graph is called its *chromatic number*.

The problem is solved with backtracking because the number of possible colour assignments is m^V, which is too large to check one by one. Backtracking builds the colouring vertex by vertex and abandons a partial colouring as soon as it breaks the rule. It is used in exam timetabling, map colouring, register allocation and frequency assignment.

Explanation

The graph is stored as an adjacency matrix, where graph[i][j] = 1 means that vertices i and j are adjacent. The vertices are coloured one at a time, starting from V0. For each vertex the colours 1 to m are tried in order, and a colour is accepted only if none of the neighbours of that vertex already has it.

If a safe colour is found, it is assigned and the algorithm moves on to the next vertex. If no colour is safe, the algorithm backtracks to the previous vertex, removes its colour and tries the next colour for it. When every vertex has been coloured, a solution is found; if the first vertex runs out of colours, no solution exists for that m.

In Figure 1, V0 gets colour 1 (red). V1 is adjacent to V0, so it gets colour 2 (green). V2 is adjacent to both V0 and V1, so only colour 3 (blue) is safe. V3 is adjacent to V0 and V2, so colours 1 and 3 are rejected and it gets colour 2. The graph is therefore coloured with 3 colours.

Diagram

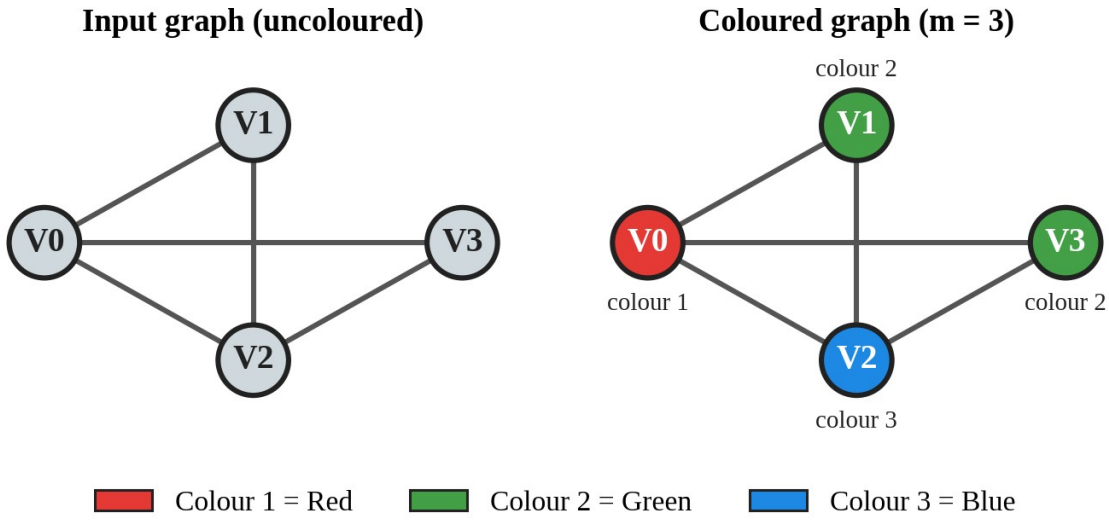
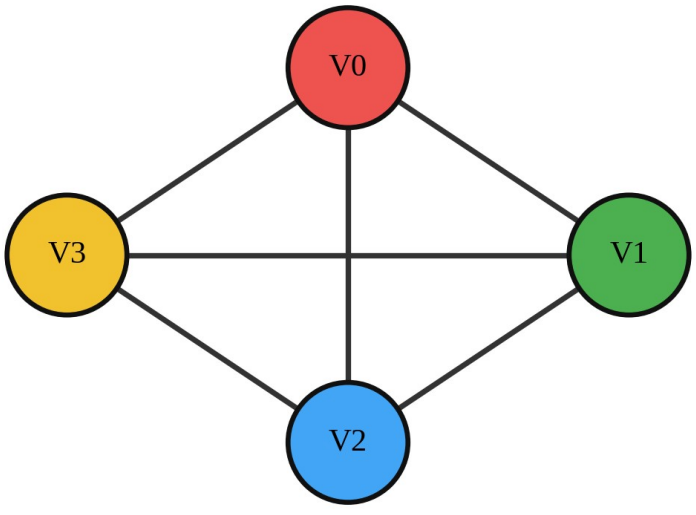


Figure 1: Graph colouring with m = 3 colours (no two adjacent vertices share a colour)

Complete Graph Example (K4)

A complete graph is a graph in which every vertex is connected to every other vertex. In K4, each of the four vertices is connected to the other three vertices.

Using backtracking, 3 colours are not enough for K4 because every pair of vertices is adjacent. Therefore, 4 colours are needed, one for each vertex.



K4: 4 colours

Figure 2: Complete graph K4 coloured with 4 colours (one colour for each vertex)

Sum of Subsets

Introduction

The Sum of Subsets problem is stated as follows: given a set of n positive integers and a target sum, find all subsets of the set whose elements add up exactly to the target. For example, in the set {5, 7, 10, 12, 15, 18, 20} with target 35, the subset {15, 20} is a solution because 15 + 20 = 35.

Checking all 2^n subsets is slow for large n. Backtracking improves on this by building each subset one element at a time and abandoning a branch as soon as it can no longer reach the target. The problem is used in resource and budget allocation and in knapsack-based cryptography.

Explanation

The numbers are first sorted. The algorithm then decides for each element, in order, whether to include it in the current subset or to skip it. This gives a binary state space tree, where the left branch means include and the right branch means skip.

While moving down the tree the running sum is tracked. If the sum equals the target, the current subset is saved as a solution. If the sum becomes greater than the target, the branch is pruned, because all numbers are positive and the sum can only grow. If all elements are used and the sum is still smaller than the target, the branch is a dead end. In both cases the algorithm backtracks by removing the last element and tries the other choice.

Figure 3 shows the state space tree for a small example, the set {3, 4, 5, 6} with target 9. Red nodes exceed the target and are pruned, grey nodes are dead ends and green nodes are solutions. The two solutions are {3, 6} and {4, 5}. Figure 4 shows the four subsets found by Program 2 for the set {5, 7, 10, 12, 15, 18, 20} with target 35.

Diagram

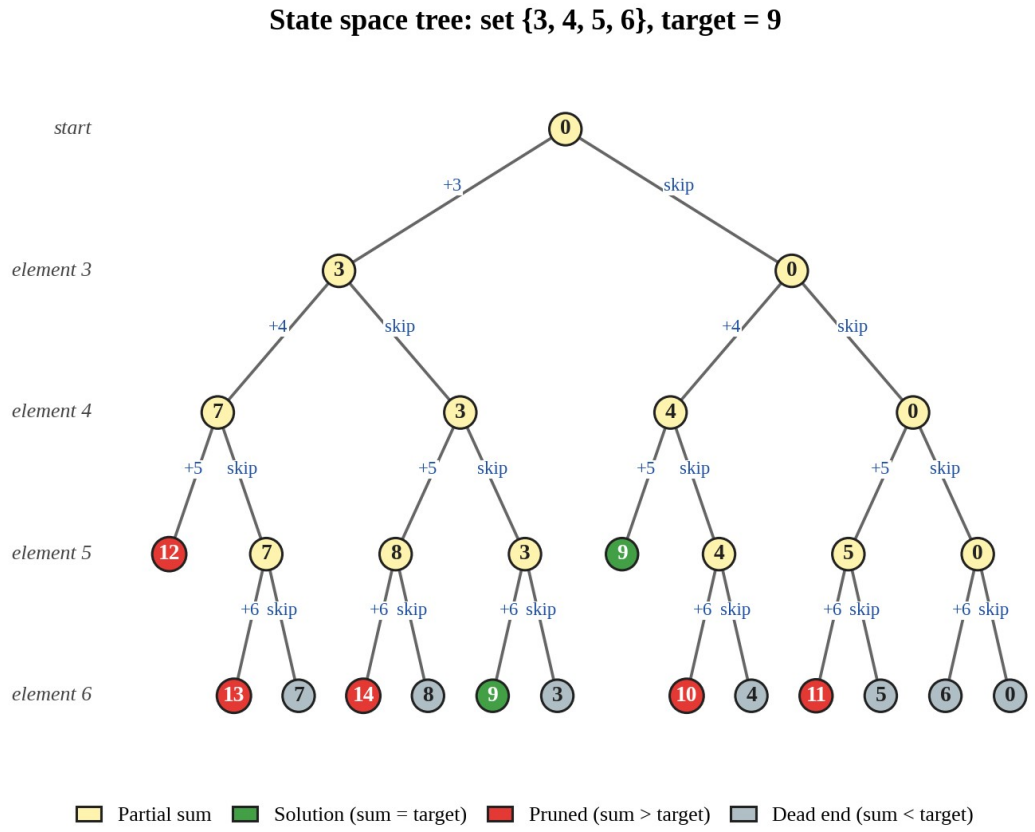


Figure 3: State space tree for the set {3, 4, 5, 6} with target 9

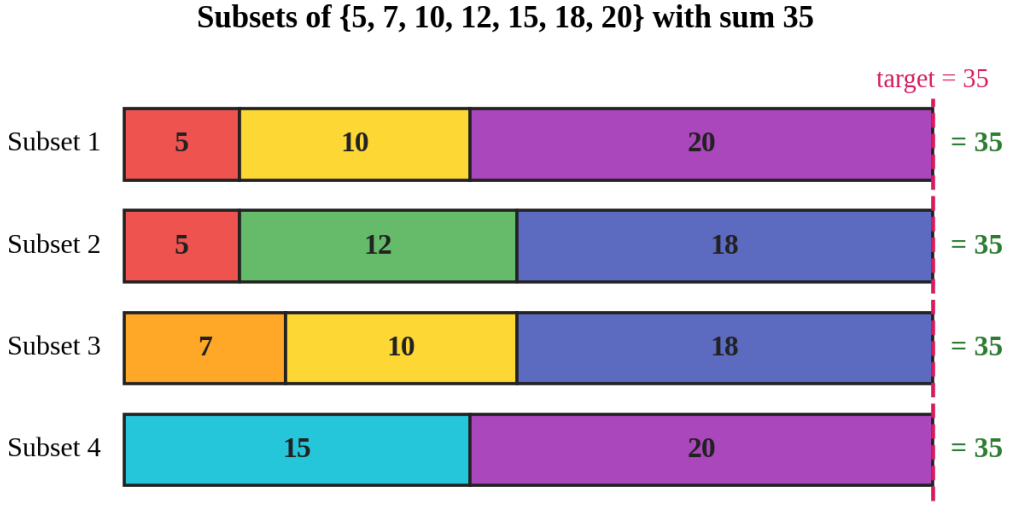


Figure 4: The four subsets of {5, 7, 10, 12, 15, 18, 20} that add up to 35 (output of Program 2)

Source Code Of The Project:

Program 1: Graph Colouring (m-Colouring Problem)

```
def is_safe(graph, colors, v, c):
    for u in range(len(graph)):
        if graph[v][u] == 1 and colors[u] == c:
            return False
    return True

def graph_coloring(graph, m, colors, v=0):
    if v == len(graph):
        return True
    for c in range(1, m + 1):
        if is_safe(graph, colors, v, c):
            colors[v] = c
            if graph_coloring(graph, m, colors, v + 1):
                return True
            colors[v] = 0  # backtrack
    return False

if __name__ == "__main__":
    graph = [
        [0, 1, 1, 1],
        [1, 0, 1, 0],
        [1, 1, 0, 1],
        [1, 0, 1, 0],
    ]
    m = 3
    colors = [0] * len(graph)
    if graph_coloring(graph, m, colors):
        print("Solution exists. Vertex colors:")
        for i, c in enumerate(colors):
            print("Vertex", i, "-> Color", c)
```

```
    else:
        print("No solution exists with", m, "colors")
```

Program 2: Sum of Subsets

```
def sum_of_subsets(nums, target, index=0, current=None, total=0, results=None):
    if current is None:
        current, results = [], []
    if total == target:
        results.append(current[:])
        return results
    if index == len(nums) or total > target:
        return results
    # Include nums[index]
    current.append(nums[index])
    sum_of_subsets(nums, target, index + 1, current, total + nums[index], results)
    current.pop()  # backtrack
    # Exclude nums[index]
    sum_of_subsets(nums, target, index + 1, current, total, results)
    return results

if __name__ == "__main__":
    nums = [10, 7, 5, 18, 12, 20, 15]
    target = 35
    nums.sort()
    answers = sum_of_subsets(nums, target)
    print("Target sum:", target)
    if answers:
        for s in answers:
            print("Subset found:", s)
    else:
        print("No subset found")
```

Step-By-Step Explanation:

A. Graph Colouring

- **Step 1 – Represent the graph:** The graph is an adjacency matrix; graph[i][j] = 1 means vertices i and j are adjacent. colors[] stores the colour (1..m) of each vertex, 0 meaning uncoloured.
- **Step 2 – is_safe():** Before giving colour c to vertex v, it checks all neighbours of v. If any neighbour already has colour c, the colour is rejected.
- **Step 3 – Try colours:** graph_coloring() tries colours 1 to m for the current vertex v and assigns the first safe one.
- **Step 4 – Recurse:** It then calls itself for vertex v+1.
- **Step 5 – Backtrack:** If the recursive call fails, colors[v] is reset to 0 and the next colour is tried. If no colour works, False is returned to the previous vertex.
- **Step 6 – Termination:** When v equals the number of vertices, every vertex is coloured and True is returned; the colours are printed.

B. Sum of Subsets

- **Step 1 – Sort the numbers:** Sorting helps prune branches earlier. The target and an empty current subset are set up.
- **Step 2 – Base case (solution):** If the running total equals the target, a copy of the current subset is saved in results.
- **Step 3 – Base case (prune):** If all elements are used or the total exceeds the target, the branch is abandoned.
- **Step 4 – Include:** The current element is added to the subset and the function recurses with the updated total.
- **Step 5 – Backtrack:** pop() removes the element, undoing the choice.
- **Step 6 – Exclude:** The function recurses again without the element. After all branches are explored, all valid subsets are returned.

Output:

Program 1: Graph Colouring

Compile Result

```
Solution exists. Vertex colors:
Vertex 0 -> Color 1
Vertex 1 -> Color 2
Vertex 2 -> Color 3
Vertex 3 -> Color 2


[Process completed - press Enter]
```

Program 2: Sum of Subsets

Compile Result

```
Target sum: 35
Subset found: [5, 10, 20]
Subset found: [5, 12, 18]
Subset found: [7, 10, 18]
Subset found: [15, 20]


[Process completed - press Enter]
```

REVIEW OF EVALUATOR :